LAB 5: STATIC CODE ANALYSIS

Student Attendance Tracker

NAME : D R Rushmitha

SRN : PES1UG24AM910

SECTION : E

1. Objective

The objective was to analyze the provided Python attendance tracker with Pylint,Bandit and Flake8, document important findings,fix security and code-quality issues,and rerun the tools to verify the changes. The program records attendance,calculates percentages,identifies shortages,exports CSV,describes a reminder job,and contains an optional ERP synchronization function.

2. Tools and method

- Pylint checks code quality,naming,documentation,and potential programming problems.
- Bandit checks Python code for common security risks.
- Flake8 checks style and formatting,including unused imports and line length.
- Reports were captured before and after changes.The updated program was run again to check its output.

3. Before-and-after results

Tool	Before	After	Result
Pylint	4.55/10	10.00/10	Target above 9.5 achieved.
Bandit	8 findings: 2 High, 3 Medium, 3 Low	No issues identified	No findings reported after fixes.
Flake8	Style issues reported	Empty report (0 bytes)	Expected if Flake8 finds no violations.

The empty Flake8 after-report is valid: Flake8 prints no text when it finds no violations. The successful command returning to the prompt with no output is consistent with this.

4. Important issues and fixes

This concise table prioritizes the high and medium severity Bandit findings and one important correctness issue.Line numbers refer to the original source before edits and may change in the updated file.

Issue	Type / severity	Original line(s)	Why it matters	Fix applied
TLS verification disabled (B501)	Security — High	60	The ERP request used verify=False, disabling certificate verification.	Removed verify=False; normal TLS certificate verification is used.
Shell command with wildcard (B609, B602, B607)	Security — High / Low	64	The backup used subprocess with shell=True and a wildcard-based tar command.	Replaced the shell command with Python pathlib and zipfile; no shell is launched.
ERP request without timeout (B113)	Security/reliability — Medium	60	The request could wait indefinitely.	Added timeout=10 and response.raise_for_status().
Insecure TLS protocol default (B503)	Security — Medium	6	The original code used the legacy TLSv1 protocol.	Changed to ssl.create_default_context().
Reminder job with shell=True (B604)	Security — Medium	73	The reminder command was represented as shell execution.	Uses a list of arguments and shell=False; the function describes the job and does not execute it.
Mutable default dictionary	Correctness/maintainability	18	A mutable default dictionary can be shared between calls.	Changed register={} to register=None and initializes it inside the function.

The before Bandit report recorded eight findings (2 High, 3 Medium, 3 Low); the after report recorded zero. Pylint improved from 4.55/10 to 10.00/10. Flake8 reported style issues before and no output after.

5. Program output comparison

5.1 Before fixes — terminal output

```
C:\Users\rushm\Downloads\SE_LAB\L5_61_attendance-tracker>python attendance_tracker.py
R01 attendance %: 66
R02 attendance %: 50
Is 75% eligible? False
Shortage list: ['R01', 'R02']
R02 long absence? True
R02 latest date: 2026-9-3
Register entries for 2026-09-01: ['R01', 'R02', 'R02']
Exported to: attendance.csv
Reminder job: {'command': 'send_sms --shortage', 'shell': True}
ERP sync skipped (no network)

C:\Users\rushm\Downloads\SE_LAB\L5_61_attendance-tracker>
```

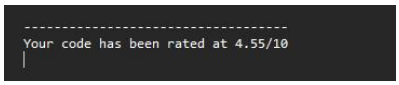
5.2 After fixes — terminal output

```
C:\Users\rushm\Downloads\SE_LAB\L5_61_attendance-tracker>python attendance_tracker.py
R01 attendance %: 66
R02 attendance %: 50
Is 75% eligible? True
Shortage list: ['R01', 'R02']
R02 long absence? True
R02 latest date: 2026-9-3
Register entries for 2026-09-01: ['R01', 'R02', 'R02']
Exported to: attendance.csv
Reminder job: {'command': ['send_sms', '--shortage'], 'shell': False}
ERP sync skipped (no network)

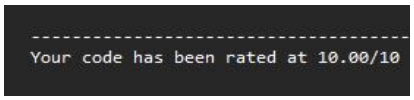
C:\Users\rushm\Downloads\SE_LAB\L5_61_attendance-tracker>
```

6. Static-analysis report evidence

6.1 Pylint — before



6.2 Pylint — after



6.3 Bandit — before

```
Code scanned:
        Total lines of code: 79
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 3
                Medium: 3
                High: 2
        Total issues (by confidence):
                Undefined: 0
                Low: 2
                Medium: 2
                High: 4
Files skipped (0):
```

6.4 Bandit — after

```
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.14.3
Run started:2026-10-27 16:09:14.136666+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 171
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

6.5 Flake8 — before

```
attendance_tracker.py:1:1: F401 'os' imported but unused
attendance_tracker.py:4:1: F401 'json' imported but unused
attendance_tracker.py:6:1: F401 'datetime.datetime' imported but unused
attendance_tracker.py:15:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:18:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:23:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:29:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:32:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:35:54: E712 comparison to False should be 'if cond is False:' or 'if not cond:'
attendance_tracker.py:41:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:43:80: E501 line too long (84 > 79 characters)
attendance_tracker.py:45:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:49:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:54:80: E501 line too long (92 > 79 characters)
attendance_tracker.py:58:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:60:80: E501 line too long (101 > 79 characters)
attendance_tracker.py:63:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:66:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:69:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:72:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:75:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:78:80: E501 line too long (87 > 79 characters)
attendance_tracker.py:81:80: E501 line too long (87 > 79 characters)
attendance_tracker.py:82:80: E501 line too long (86 > 79 characters)
attendance_tracker.py:96:1: E305 expected 2 blank lines after class or function definition, found 1
```

6.6 Flake8 — after

The Flake8 after-report is empty because all Python code style and formatting issues were corrected. When Flake8 was run again, it did not detect any remaining violations.

7. Reflection Questions and answers :

1.What did you learn from static analysis?

Ans : Static analysis tools can identify problems before a program is deployed.Pylint helped identify code-quality issues,Bandit highlighted security risks,and Flake8 identified formatting and unused import issues.

2.Which finding was most important and why?

Ans : The high severity Bandit findings were important because disabling TLS certificate verification and using shell=True with a wildcard command can expose a program to security risks.The code was changed to use normal TLS verification and Python's zipfile APIs instead of a shell command.

3.Were all warnings equally important?

Ans : No.Security findings and reliability issues such as missing network timeouts deserve priority over style issues.Style still matters because readable,consistent code is easier to maintain.

4.How will you use static analysis in future work?

Ans : I will run analysis tools early, prioritize high and medium severity findings,keep before and after reports,rerun the tools after changes,and check that the program's behavior remains sensible.

8. Conclusion

This lab demonstrated how static code analysis tools help improve Python code quality,security,and readability.Pylint, Bandit, and Flake8 were used to identify issues in the original attendance tracker program.The identified issues were addressed by improving coding practices,fixing formatting problems, and replacing insecure operations with safer alternatives.After making the changes,the final checks achieved a Pylint score of 10.00/10,zero Bandit findings,and no Flake8 violations.Overall,this lab helped develop practical skills in identifying,fixing,and verifying code issues using static analysis tools,resulting in code that is more secure,maintainable,and easier to understand.